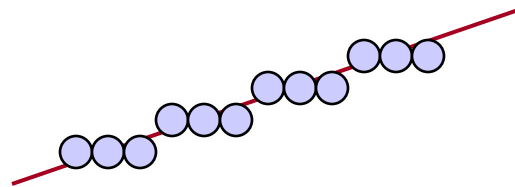
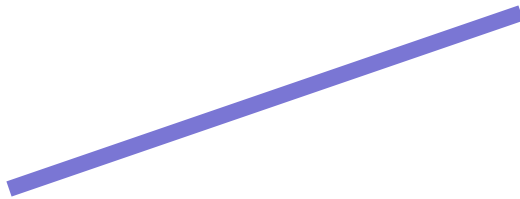


Line and Curve Drawing Algorithms

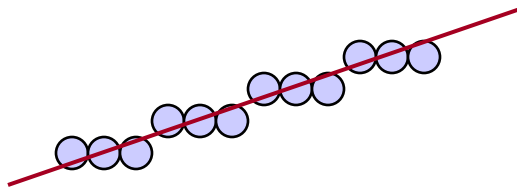
Line Drawing



Line Drawing Algorithms

- 1. Using line equation*
- 2. DDA algorithm*
- 3. Bresenham's algorithm*

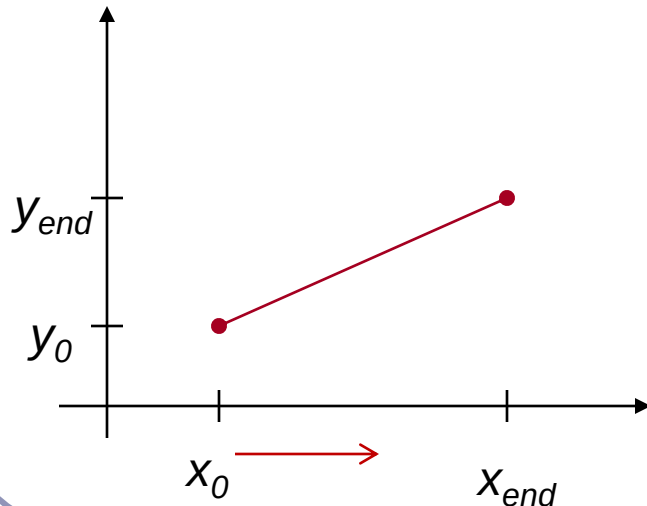
Using Line Equation



$$|m| < 1$$

$$y = m.x + b$$

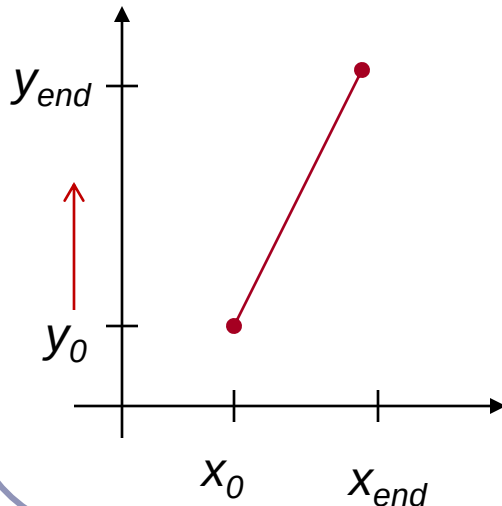
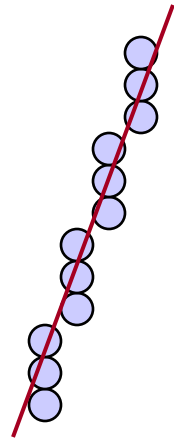
Round y



$$m = \frac{y_{end} - y_0}{x_{end} - x_0}$$

$$b = y_0 - m.x_0$$

Using Line Equation



$$|m| > 1$$

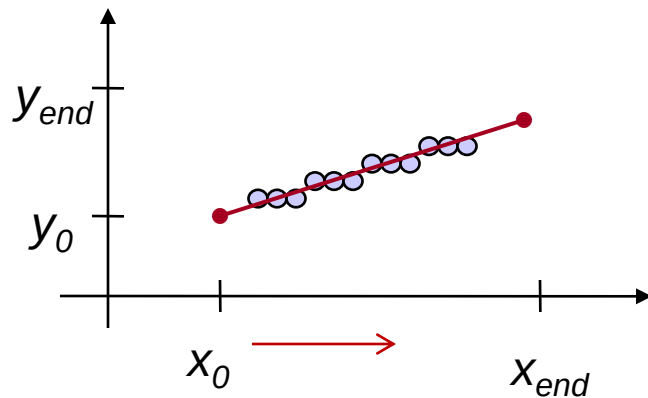
$$x = \frac{(y - b)}{m}$$

Round x

$$m = \frac{y_{end} - y_0}{x_{end} - x_0}$$

$$b = y_0 - m \cdot x_0$$

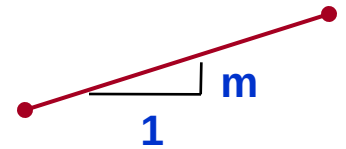
DDA Algorithm



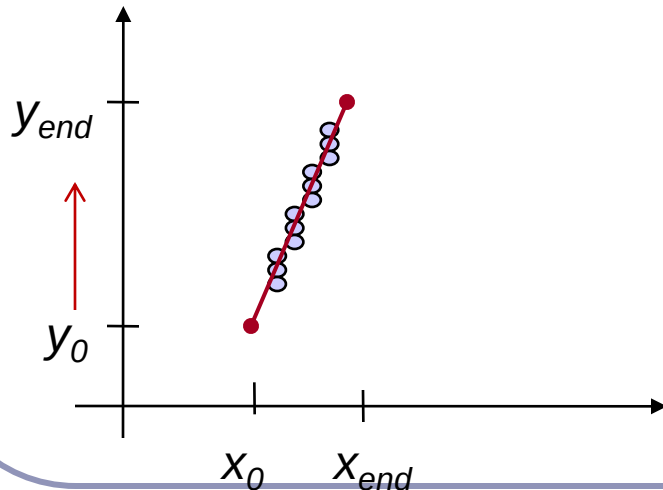
if $|m| < 1$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + m$$



Round y



if $|m| > 1$

$$y_{k+1} = y_k + 1$$

$$x_{k+1} = x_k + 1/m$$

Round x

DDA Algorithm

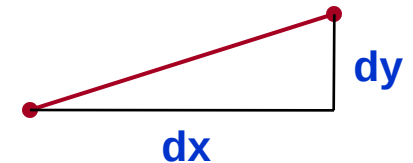
```
#include <stdlib.h>
#include <math.h>

inline int round (const float a) { return int (a + 0.5); }

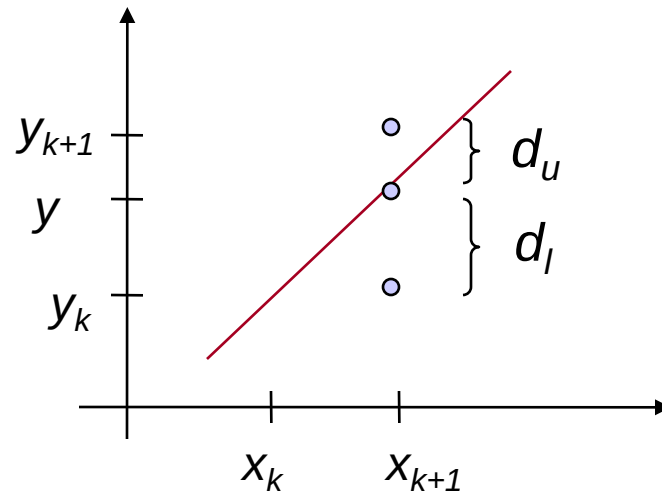
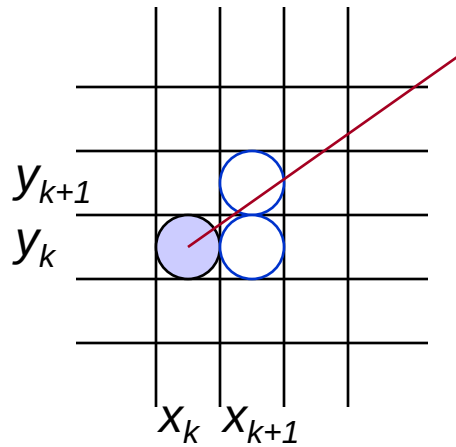
void lineDDA (int x0, int y0, int xEnd, int yEnd)
{
    int dx = xEnd - x0,  dy = yEnd - y0,  steps,  k;
    float xIncrement, yIncrement, x = x0, y = y0;

    if (fabs (dx) > fabs (dy))
        steps = fabs (dx);          /* |m|<1 */
    else
        steps = fabs (dy);          /* |m|>=1 */
    xIncrement = float (dx) / float (steps);
    yIncrement = float (dy) / float (steps);

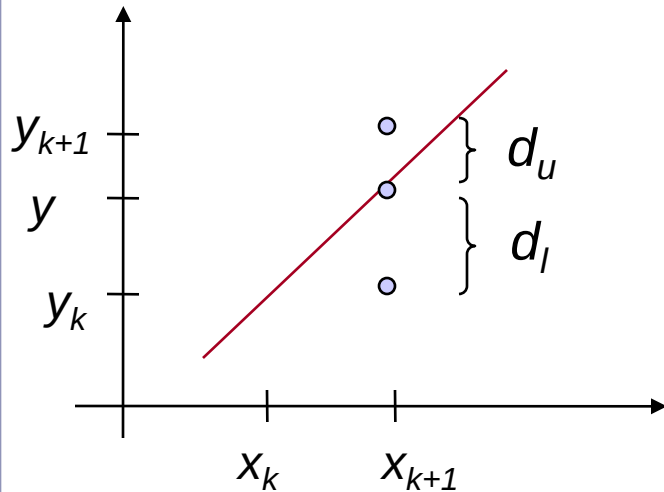
    setPixel (round (x), round (y));
    for (k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        setPixel (round (x), round (y));
    }
}
```



Bresenham's Line Algorithm



Bresenham's Line Algorithm



$$y = m.x_{k+1} + b$$

$$d_l = y - y_k = m.x_{k+1} + b - y_k$$

$$d_u = y_{k+1} - y = y_{k+1} - m.x_{k+1} - b \quad y_{k+1} = y_k + 1$$

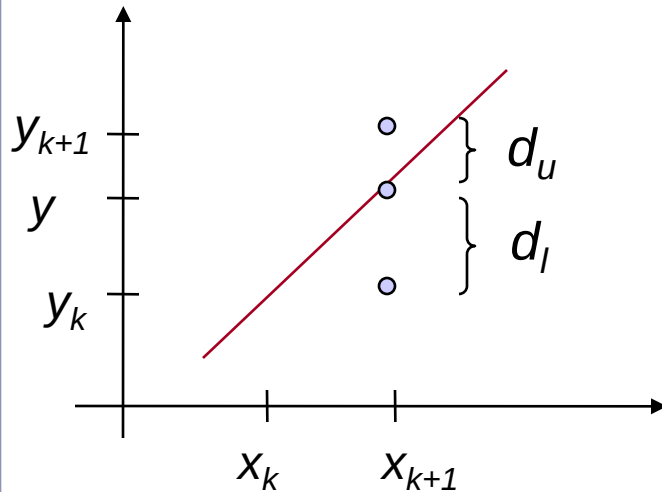
$$\begin{aligned} d_l - d_u &= 2.m.x_{k+1} - 2.y_k + 2.b - 1 \\ &= 2 \frac{\Delta y}{\Delta x} (x_k + 1) - 2.y_k + 2.b - 1 \end{aligned}$$

$$P_k < 0 \Rightarrow d_u > d_l \Rightarrow y_k$$

$$P_k > 0 \Rightarrow d_l > d_u \Rightarrow y_{k+1}$$

$$\begin{aligned} P_k &= \Delta x(d_l - d_u) \\ &= 2.\Delta y.x_k - 2.\Delta x.y_k + 2.\Delta y + \Delta x(2.b - 1) \\ &= 2.\Delta y.x_k - 2.\Delta x.y_k + C \end{aligned}$$

Bresenham's Line Algorithm



$$P_k = 2.\Delta y.x_k - 2.\Delta x.y_k + C$$

$$P_{k+1} = 2.\Delta y.x_{k+1} - 2.\Delta x.y_{k+1} + C$$

$$\begin{aligned} P_{k+1} - P_k &= 2.\Delta y(x_{k+1} - x_k) - 2.\Delta x(y_{k+1} - y_k) \\ &= 2.\Delta y - 2.\Delta x(y_{k+1} - y_k) \end{aligned}$$

$$P_{k+1} = P_k + 2.\Delta y - 2.\Delta x(y_{k+1} - y_k)$$

$$P_0 = 2.\Delta y - \Delta x$$

if $P_k < 0$ plot (x_{k+1}, y_k) and $P_{k+1} = P_k + 2.\Delta y$

if $P_k > 0$ plot (x_{k+1}, y_{k+1}) and $P_{k+1} = P_k + 2.\Delta y - 2.\Delta x$

Bresenham's Line Algorithm

```
#include <stdlib.h>
#include <math.h>
/* Bresenham line-drawing procedure for |m|<1.0 */

void lineBres (int x0, int y0, int xEnd, int yEnd)
{
    int dx = fabs(xEnd - x0),
        dy = fabs(yEnd - y0);
    int p = 2 * dy - dx;
    int twoDy = 2 * dy,
        twoDyMinusDx = 2 * (dy - dx);
    int x, y;

    /* Determine which endpoint to use as start
       position.  */
    if (x0 > xEnd) {
        x = xEnd;
        y = yEnd;
        xEnd = x0;
    }
```

```
else {
    x = x0;
    y = y0;
}
setPixel (x, y);

while (x < xEnd) {
    x++;
    if (p < 0)
        p += twoDy;
    else {
        y++;
        p += twoDyMinusDx;
    }
    setPixel (x, y);
}
```

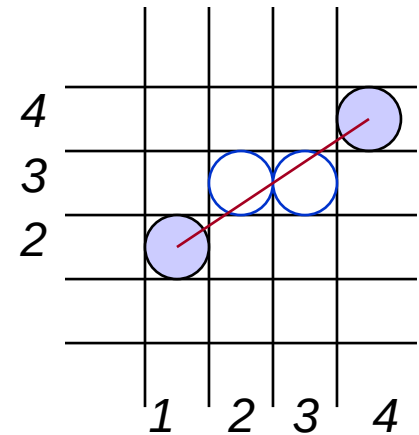
Bresenham's Line Algorithm

if $P_k < 0$ plot (x_{k+1}, y_k) and $P_{k+1} = P_k + 2.\Delta y$
if $P_k > 0$ plot (x_{k+1}, y_{k+1}) and $P_{k+1} = P_k + 2.\Delta y - 2.\Delta x$

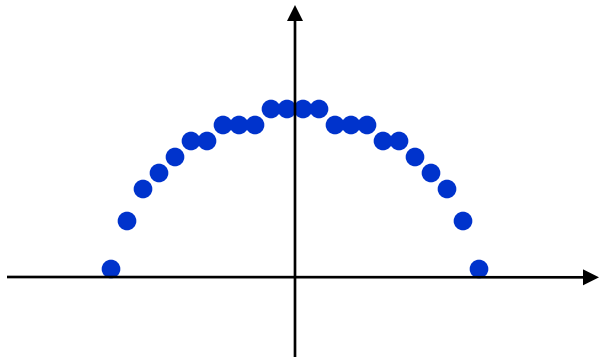
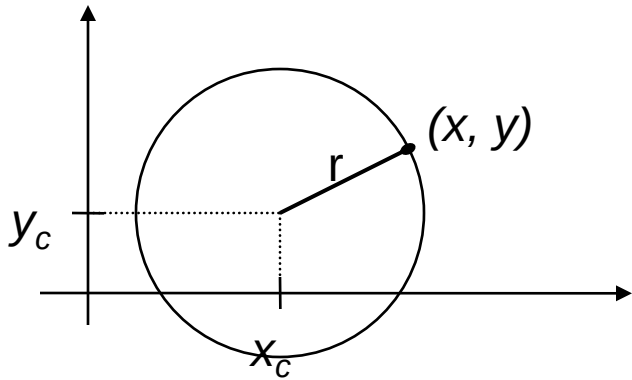
$$\begin{aligned}\Delta x &= 4 - 1 = 3 & 2.\Delta y &= 4 & P_0 &= 2.\Delta y - \Delta x = 1 \\ \Delta y &= 4 - 2 = 2 & 2.\Delta y - 2.\Delta x &= -2\end{aligned}$$

$$P_0 > 0 \text{ plot } (2, 3) \quad P_1 = P_0 + 2.\Delta y - 2.\Delta x = -1$$

$$P_1 < 0 \text{ plot } (3, 3) \quad P_2 = P_1 + 2.\Delta y = 3$$



Circle Drawing



Pythagorean Theorem:

$$x^2 + y^2 = r^2$$

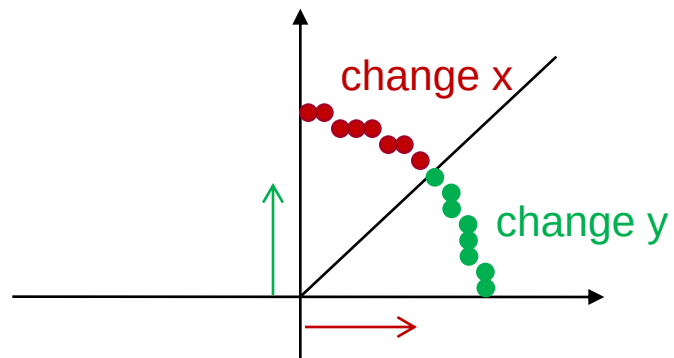
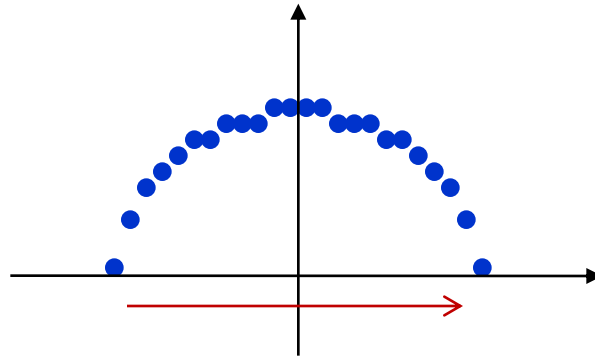
$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

$$(x_c - r) \leq x \leq (x_c + r)$$

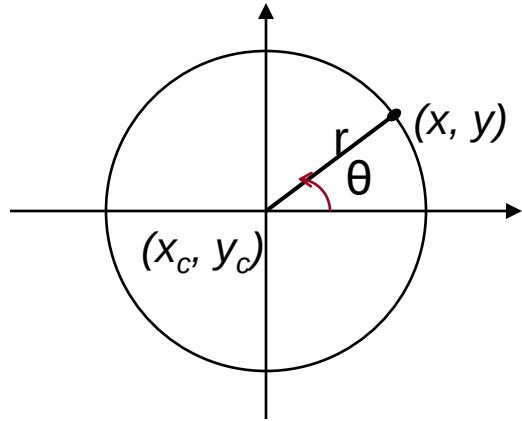
$$y = y_c \pm \sqrt{r^2 - (x - x_c)^2}$$

Round y

Circle Drawing



Circle Drawing using polar coordinates

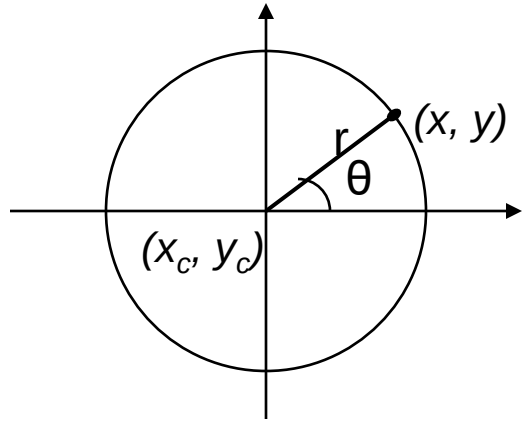


$$x = x_c + r \cdot \cos \theta$$

$$y = y_c + r \cdot \sin \theta$$

*change θ with step
size $1/r$*

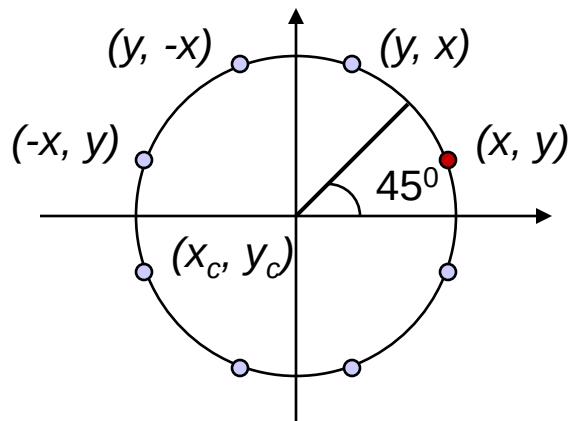
Circle Drawing using polar coordinates



$$x = x_c + r \cdot \cos \theta$$

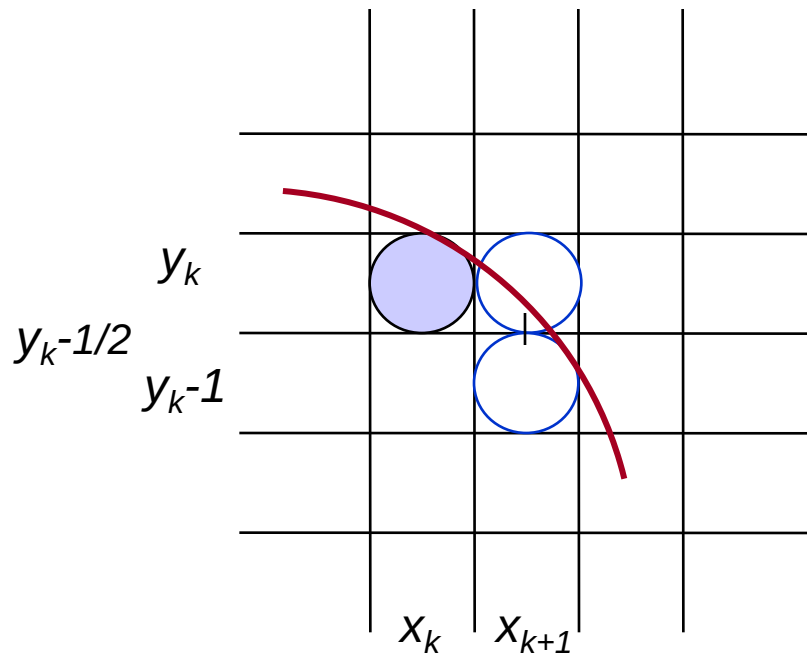
$$y = y_c + r \cdot \sin \theta$$

*change θ with step
size $1/r$*



use symmetry if $\theta > 45^\circ$

Midpoint Circle Algorithm



$$f(x,y) = x^2 + y^2 - r^2$$

$$f(x,y) \begin{cases} < 0 & \text{if } (x,y) \text{ is inside circle} \\ = 0 & \text{if } (x,y) \text{ is on the circle} \\ > 0 & \text{if } (x,y) \text{ is outside circle} \end{cases}$$

use symmetry if $x > y$

Midpoint Circle Algorithm

```
#include <GL/glut.h>

class scrPt {
public:
    GLint x, y;
};

void setPixel (GLint x, GLint y)
{
    glBegin (GL_POINTS);
    glVertex2i (x, y);
    glEnd ( );
}

void circleMidpoint (scrPt circCtr, GLint radius)
{
    scrPt circPt;

    GLint p = 1 - radius;
    circPt.x = 0;
    circPt.y = radius;
    void circlePlotPoints (scrPt, scrPt);

    /* Plot the initial point in each circle quadrant. */
    circlePlotPoints (circCtr, circPt);

    /* Calculate next points and plot in each octant. */
    while (circPt.x < circPt.y) {
        circPt.x++;
        if (p < 0)
            p += 2 * circPt.x + 1;
        else {
            circPt.y--;
            p += 2 * (circPt.x - circPt.y) + 1;
        }
        circlePlotPoints (circCtr, circPt);
    }
}

void circlePlotPoints (scrPt circCtr, scrPt circPt);
{
    setPixel (circCtr.x + circPt.x, circCtr.y + circPt.y);
    setPixel (circCtr.x - circPt.x, circCtr.y + circPt.y);
    setPixel (circCtr.x + circPt.x, circCtr.y - circPt.y);
    setPixel (circCtr.x - circPt.x, circCtr.y - circPt.y);
    setPixel (circCtr.x + circPt.y, circCtr.y + circPt.x);
    setPixel (circCtr.x - circPt.y, circCtr.y + circPt.x);
    setPixel (circCtr.x + circPt.y, circCtr.y - circPt.x);
    setPixel (circCtr.x - circPt.y, circCtr.y - circPt.x);
}
```

OpenGL

```
#include <GL/glut.h>          // (or others, depending on the system in use)

void init (void)
{
    glClearColor (1.0, 1.0, 1.0, 0.0); // Set display-window color to white.

    glMatrixMode (GL_PROJECTION);       // Set projection parameters.
    gluOrtho2D (0.0, 200.0, 0.0, 150.0);

}

void lineSegment (void)
{
    glClear (GL_COLOR_BUFFER_BIT); // Clear display window.

    glColor3f (0.0, 0.0, 1.0);        // Set line segment color to blue.
    glBegin (GL_LINES);
        glVertex2i (180, 15);         // Specify line-segment geometry.
        glVertex2i (10, 145);
    glEnd ( );

    glFlush ( ); // Process all OpenGL routines as quickly as possible.
}

void main (int argc, char** argv)
{
    glutInit (&argc, argv);           // Initialize GLUT.
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB); // Set display mode.
    glutInitWindowPosition (50, 100);  // Set top-left display-window position.
    glutInitWindowSize (400, 300);     // Set display-window width and height.
    glutCreateWindow ("An Example OpenGL Program"); // Create display window.

    init ( );                          // Execute initialization procedure.
    glutDisplayFunc (lineSegment);     // Send graphics to display window.
    glutMainLoop ( );                 // Display everything and wait.
}
```

OpenGL

Point Functions

- `glVertex* ( ) ;`
 - `* : 2, 3, 4`
 - `i (integer)`
 - `s (short)`
 - `f (float)`
 - `d (double)`

Ex:

```
glBegin(GL_POINTS);  
    glVertex2i(50, 100);  
glEnd();
```

```
int p1[ ]={50, 100};  
glBegin(GL_POINTS);  
    glVertex2iv(p1);  
glEnd();
```

OpenGL

Line Functions

- **GL_LINES**
- **GL_LINE_STRIP**
- **GL_LINE_LOOP**

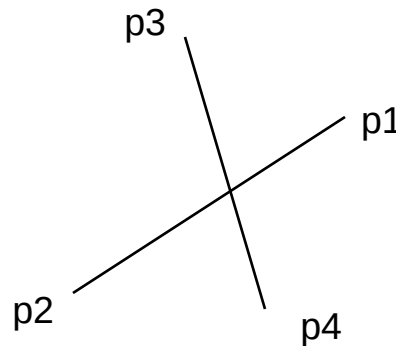
Ex:

```
glBegin(GL_LINES) ;  
    glVertex2iv(p1) ;  
    glVertex2iv(p2) ;  
glEnd() ;
```

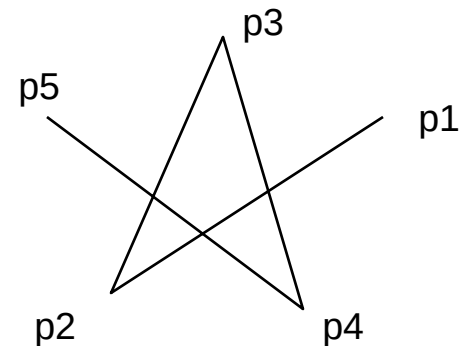
OpenGL

```
glBegin(GL_LINES);  
    glVertex2iv(p1);  
    glVertex2iv(p2);  
    glVertex2iv(p3);  
    glVertex2iv(p4);  
    glVertex2iv(p5);  
glEnd();
```

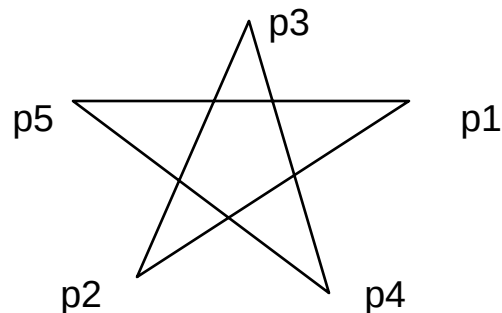
GL_LINES



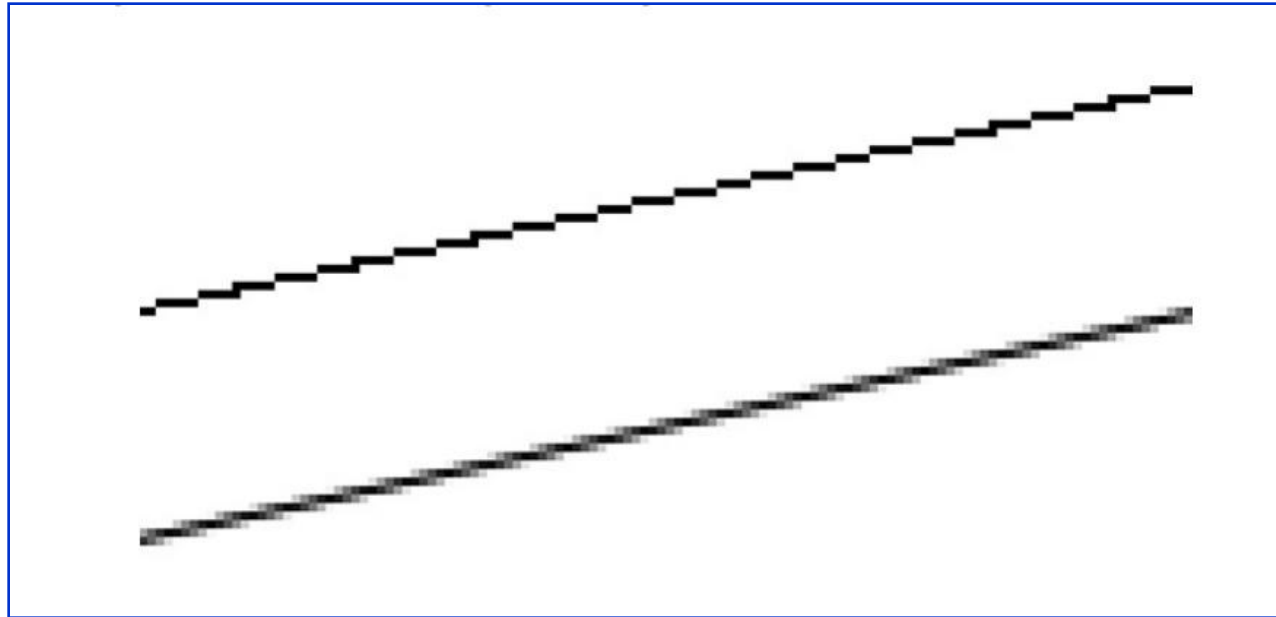
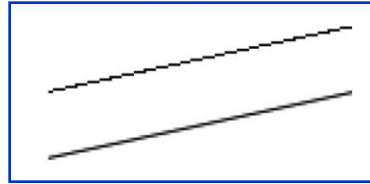
GL_LINE_STRIP



GL_LINE_LOOP



Antialiasing

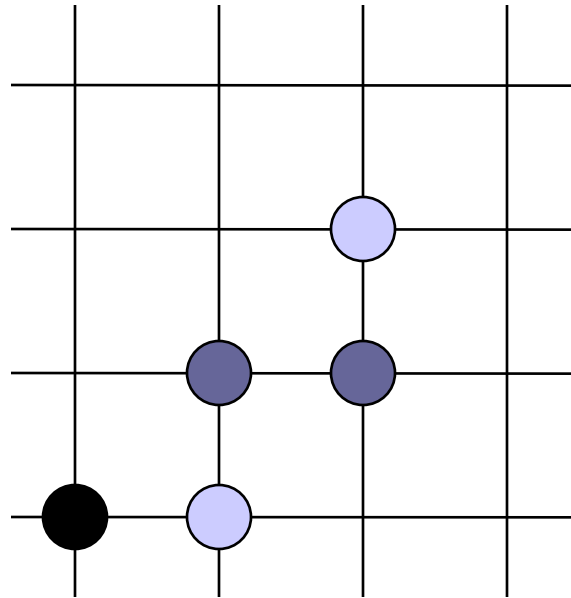
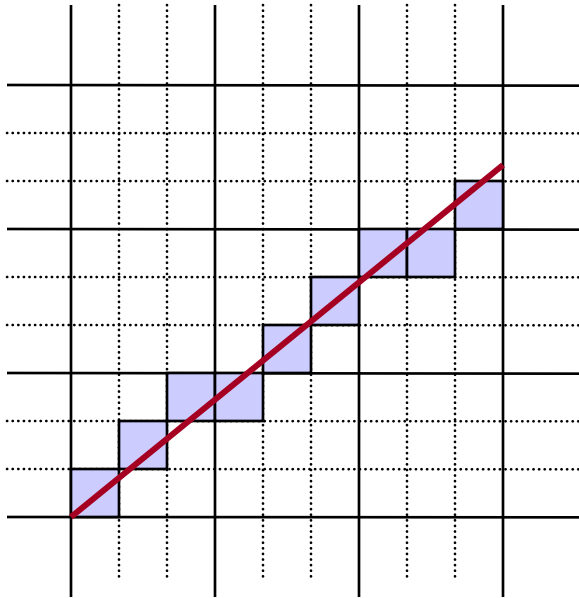


No Antialiasing

With Antialiasing

Antialiasing

Supersampling



Count the
number of
subpixels
that overlap
the line path.

Set the intensity
proportional
to this count.

Antialiasing

Supersampling

1	2	1
2	4	2
1	2	1

3x3 Virtual Pixels

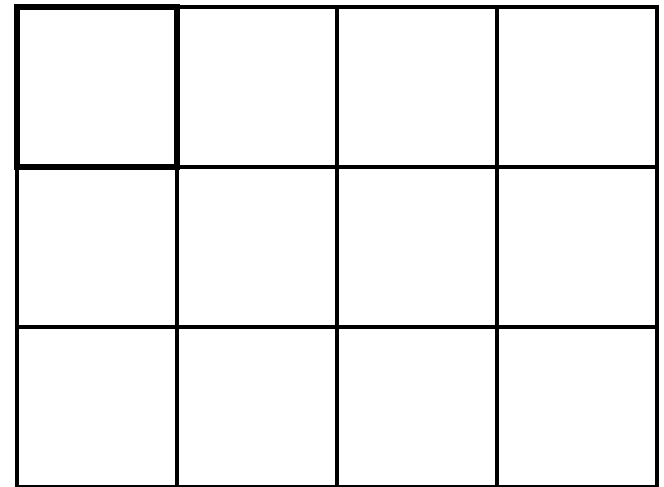
(255,255,255) (255,255,255) (255,0,0)

(255,255,255) (255,0,0) (255,255,255)

(255,0,0) (255,255,255) (255,255,255)

Example

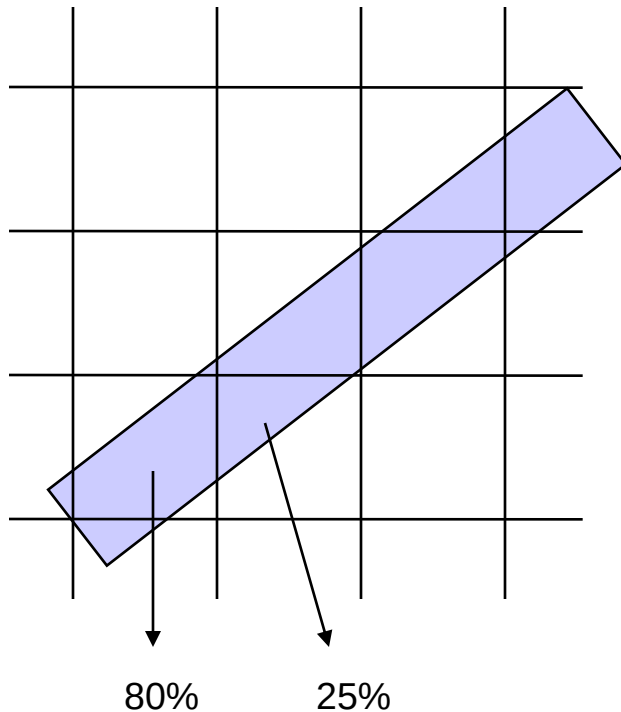
(255, 159, 159)



Actual Screen Pixels

Antialiasing

Area Sampling



Line is treated as a rectangle.

Calculate the overlap areas for pixels.

Set intensity proportional to the overlap areas.

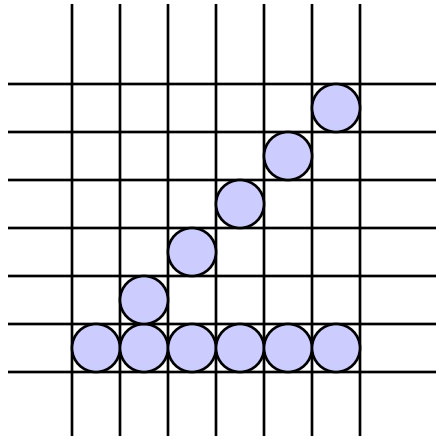
Antialiasing

Pixel Sampling

Micropositioning

Electron beam is shifted $\frac{1}{2}$, $\frac{1}{4}$, $\frac{3}{4}$
of a pixel diameter.

Line Intensity differences



Change the line drawing algorithm:

- For horizontal and vertical lines use the lowest intensity
- For 45° lines use the highest intensity